# 14.1 SPINAL：不部署就别谈模型效果提升怎么回事？

2026-01-23 09:00

阅读时长：20分钟

我们之前在讲工具调用的模型训练时候，大多用的是强化学习（以及变种）这类，这种训练方式有效的前提条件是模型能够在环境中实际执行工具调用（比如在 SWE-bench 或 Terminal-Bench 上），然后根据结果来调整策略。

**这种训练方式的核心问题是：**

- 需要在真实环境中反复执行工具调用，成本高昂
- 模型能学会的工具调用能力，很大程度上受限于训练环境提供的工具和数据，很难泛化到没见过的工具上
- 这个方法对小模型效果不佳，因为小模型在探索阶段会产生大量无效或错误的工具调用，浪费计算资源

**我们这篇文章介绍的SPINAL方法，则是以这种不同方式，来提升工具调用能力：**

- 不需要实际执行工具调用，只需要监督信号（ground truth）
- 能够生成结构化、可解释的诊断报告，指出模型在哪里、为什么、怎么改
- 在多个工具调用基准上，实现了显著的性能提升，而且模型规模越小，提升效果越明显

**SPINAL 的核心贡献：**

- 提出一个新的评估范式：将工具调用的评估，从单纯的结果导向，扩展到包括诊断、定位、解释、修复建议的全流程
- 发布 BLUE 数据集：包含 1,000 多个精心筛选的多轮对话，覆盖多种工具调用场景
- 开发 SPINAL 模型：在 4B 参数量级，达到 GPT-4.1 的性能水平，且对模型规模的扩展表现出良好的可扩展性

**这篇文章，我们会从：**

- 为什么现有的工具调用评估方法不够用？
- 工具调用中，最常见的错误模式是什么？
- BLUE 数据集是怎么构建的？
- SPINAL 方法的三个关键设计：Plan-Ground-Reflect 范式、Schema-Agnostic 建模、层级奖励设计
- 实验结果告诉我们哪些关于模型规模、泛化能力、数据质量的重要结论

## 为什么现有评估方法不够用？

### 现有工具调用评估的三个问题

- 成本高昂：需要在真实环境中执行工具调用，对于多轮对话、需要外部 API 响应的场景，成本和延迟都是问题
- 覆盖有限：只能测试已有工具和已有场景，很难评估模型在新工具、新组合上的泛化能力
- 信号稀疏：只有最终结果（成功/失败），没有中间过程的反馈，很难定位具体哪个环节出了问题

### 我们的观察：错误模式的层级结构

工具调用任务，可以分解为三个阶段：

- 意图理解：理解用户需求，决定是否需要调用工具、调用哪个工具
- 参数填充：从对话上下文中，提取出调用工具所需的参数
- 结果解释：根据工具返回的结果，决定下一步是继续调用工具、还是直接回答用户

这三个阶段，对应三种不同的错误模式：

- 意图理解错误：该调用工具的时候，模型直接回答了（幻觉）
- 参数填充错误：工具选对了，但参数写错了（格式错误、缺少必填项、从对话中提取错误的值）
- 结果解释错误：工具返回了正确结果，但模型没有正确理解、或忽略了关键信息

这种层级结构，意味着：

- 评估不能只看最终结果，需要能够定位具体是哪个阶段出了问题
- 训练信号也应该分层设计，对不同阶段的错误，给出不同的反馈

### 现有数据集的不足

- BFCL：提供了比较全面的 LLM 工具调用评估基准，但数据集主要是单轮对话、或简单的多轮对话，很难测试模型在复杂、长程交互中的表现
- ToolBench：规模很大，但很多样本是合成的，质量参差不齐，很难保证评估的可信度
- 真实对话：最接近实际应用场景，但隐私问题、标注成本、可复现性都是挑战

### 我们的选择：合成多轮对话

为了能够精准地测试模型在各个环节的表现，我们需要一个高质量、可控的、覆盖多种错误模式的评估集。

所以，我们选择了合成多轮对话的方式，并设计了一个多阶段、多模型协同的数据生成流程，来保证数据的质量和覆盖度。

## BLUE 数据集

### 为什么需要一个新的数据集？

BLUE 是我们为了工具调用评估，专门构建的数据集。

核心特点：

- 多轮对话：平均 5 轮左右，最长可达 10 轮，覆盖单轮、顺序调用、并行调用、澄清、拒绝等多种交互模式
- 多工具环境：平均 6 个工具可选，包括搜索、计算、日历、天气、文件操作、数据库查询等常见 API 类型
- 合成数据：通过多模型协同生成，保证覆盖多样性和质量可控
- 开放格式：兼容 ShareGPT 格式，方便社区使用和扩展

### 与现有数据集的对比

| 数据集 | 多轮 | 多工具 | 真实执行 | 推理标注 |
|---|---|---|---|---|
| BFCL | 部分 | 是 | 否 | 否 |
| ToolBench | 否 | 是 | 否 | 否 |
| 真实对话 | 是 | 不定 | 是 | 不定 |
| BLUE | 是 | 是 | 否 | 是 |

BLUE 的核心优势：在保证多样性、可控性、可复现性的前提下，尽可能地接近真实对话的复杂度。

### 数据来源

BLUE 由三部分组成：

- 人工精选种子：从 ToolBench 的高质量子集、以及我们内部设计的场景出发，人工筛选出一批高质量、覆盖典型场景的多轮对话，作为生成的参考模板
- 多模型协同生成：用 GPT-4.1 扮演用户、Claude-Sonnet-4 扮演助手、Qwen-Plus 扮演工具，三个模型分别扮演不同角色，生成对话
- 人工审核与迭代：对生成的对话，进行人工审核，剔除质量不高的样本，并根据问题调整生成提示词

### 工具池设计

- 工具数量：每条对话，可用工具池大小在 6 个左右
- 工具类型：覆盖搜索、计算、日期时间、天气、文件操作、数据库查询、邮件、日历等常见 API 类型
- 干扰项：工具池中，除了必要的工具，还会加入一些相关但不需要用的工具，考验模型的选择能力
- 格式：工具的 schema 遵循 OpenAI function calling 格式，保证与主流 API 的兼容性

### 质量控制

- 人工抽检：生成的对话，人工抽检一部分，检查逻辑是否合理、标注是否准确
- 模型辅助过滤：用 GPT-4.1 对生成的对话，进行一轮自动打分，低分的样本优先进入人工审核
- 迭代优化：根据模型在 BLUE 上的表现，反过来分析数据的不足，指导下一轮数据的生成方向

## SPINAL 模型

### 核心思路：从"结果"到"诊断"

传统的工具调用训练，只关注最终结果（成功/失败）。

SPINAL 的目标，是让模型学会，对自己的输出，进行诊断和修复。

这个能力，对于小模型尤其重要——小模型容易犯很多低级错误，如果有一个"检查清单"来提醒它，很多问题就可以避免。

### 三大设计原则

- 分层奖励：不同阶段的错误，给出不同粒度的奖励
- 结构化输出：诊断报告，要能被程序解析，方便自动化处理
- 可解释性：诊断报告，要能被人类理解，方便开发者调试

### Plan-Ground-Reflect 范式

SPINAL 采用一个三阶段的生成流程：

- Plan（规划）：模型首先，分析用户意图，决定需要调用哪些工具、以什么顺序
- Ground（锚定）：模型，将每个工具调用，与具体的参数、上下文信息，进行对应
- Reflect（反思）：模型，对整个工具调用序列，进行自我检查，发现潜在的问题、给出修复建议

这个流程的核心思想：先规划、再执行、最后反思，模仿人类专家的工作流程。

### Schema-Agnostic 建模

SPINAL 的设计，不依赖特定的工具 schema，而是通过以下方式，来保证泛化能力：

- 工具 schema 作为输入：模型，接收工具的 schema 作为输入，而不是把 schema 硬编码进模型
- 结构化输出：模型，输出一个通用的、可解析的诊断格式，不绑定任何特定工具

这样，模型，学到的，是"如何诊断工具调用"的通用能力，而不是"如何调用某个特定工具"的具体知识。

### 层级奖励设计

SPINAL 采用三个层级的奖励：

- 格式奖励：诊断报告，是否可解析？
- 定位奖励：指出的错误，是否与真实错误，位置一致？
- 内容奖励：诊断报告，对修复建议的，预测是否正确？

这个设计，鼓励模型，先学会，输出合法的格式，再学习，准确定位问题，最后，才是，提高修复建议的质量。

### 为什么这个设计，对小模型尤其有效？

小模型，容易犯很多低级错误，比如格式错误、漏掉必填参数。

SPINAL 的分层奖励，会先，教会，它，这些基本功，再，逐步，提升，更高，层级，的，能力。

这，比，直接，用，一，个，大，模型，来，蒸，馏，，，效，果，，，